Kafka

Assignment

Problem Statement:

You are required to build a hands-on, end-to-end Restaurant Franchise Order Event Processing System using Apache Kafka, Docker, and Kafka CLI commands.

Your task is to implement a simple event-driven application where order events created from a command-line producer are published to Kafka and consumed independently by multiple downstream services.

Scenario:

A restaurant franchise with two outlets, RESTAURANT-101 and RESTAURANT-102, needs a CLI-based Kafka system where order events are published to a single topic, restaurant.orders.created, using restaurant_id as the key so orders from the same outlet always go to the same partition. These events must be consumed independently by Kitchen Service, Inventory Service, and Customer Notification Service, while demonstrating Kafka concepts such as producers, consumers, topics, partitions, consumer groups, replay, and failure isolation.

Requirements

1. Project Setup

- Create the project using any programming language of your choice.
- Install Docker on your system so you can run Kafka inside containers instead of setting it up manually on your machine.
- Install Docker Compose so you can define and start the Kafka environment using a single `docker-compose.yml` file.

2. Kafka Environment Setup

- In the `docker-compose.yml` file, include the Kafka image, container name, port mappings, required environment variables, and any supporting service needed for Kafka to start properly.
- Start the Kafka environment using Docker Compose and confirm that the broker is running successfully.
- After Kafka starts successfully, create the topic `restaurant.orders.created` using Kafka CLI commands.
- Set up the topic with multiple partitions so messages can be distributed properly based on the key.

3. Event Model

- Create restaurant order events with the following fields:
    - order_id
    - restaurant_id
    - customer_name

    - items
    - order_time
    - order_status

4. Kafka Configuration

- Configure Kafka connection details in the application.
- Use `restaurant_id` as the message key.
- Show that orders from the same restaurant consistently go to the same partition.

5. Producer Implementation

- Create a CLI-based Order Producer application.
- The producer should allow users to create restaurant orders through:
    - command-line arguments.
- Produce events for both RESTAURANT-101 and RESTAURANT-102

6. Consumer Implementation

- Create the following consumers:
    - Kitchen Service
    - Inventory Service
    - Customer Notification Service
- Each consumer must:
    - subscribe to the same topic
    - process the same order events independently
    - belong to its own consumer group

7. Failure Isolation and Recovery

- Stop one consumer temporarily
- Continue producing order events while that consumer is down
- Show that the other consumers continue to work.
- Restart the stopped consumer and show that it resumes from its last committed offset.

8. Replay Demonstration

- Reset offsets for one consumer group using Kafka CLI..
- Reprocess old events from the topic

9. Add README file

- Add a `README.md` file to the project that clearly explains how to start Kafka, how to create the topic, how to run the producer and consumers, and how to demonstrate failure recovery and replay.

Deliverables:

- **GitHub PR Link:** Push the complete codebase to a private RSL GitHub repository (share the access with your trainer). Trainees must share the Pull Request (PR) link and the branch name. Also, make sure the codebase contains the README.md with the instructions and guide to run the program.

- **Screen Recording:** Include a short demo video showing the complete working flow of the project. The video should demonstrate all three consumers processing the same events in separate consumer groups, consistent partitioning based on `restaurant_id`, failure isolation when one consumer is stopped, recovery from the last committed offset after restart, and replay of old events by resetting offsets.
- **Email Status Updates:** Trainees must share their regular status updates over email.